

Decision Trees Unraveled: Analyzing Tic-Tac-Toe Endgame Scenarios (June 2024)

Aayush Man Shrestha¹ and Pradeepti Dongol²

Department of Electronics and Computer Engineering, IOE, Thapathali Campus, Kathmandu 44600, Nepal

Corresponding author: Aayush Man Shrestha (aayush.m.shrestha@gmail.com)

ABSTRACT Decision trees are powerful tools in machine learning that facilitate intuitive decision-making processes. Decision trees are powerful tools in machine learning that facilitate intuitive decision-making processes by partitioning data based on feature attributes. This report aims to construct decision trees that model the decision-making process in tic-tac-toe when the player plays as X. Different decision trees are explored by varying parameters such as splitting criteria, pruning strategies, and conducting feature importance analysis. Their performances are compared to assess effectiveness in capturing optimal strategy. Using the Classification and Regression Trees (CART) algorithm implemented in sklearn, our analysis predicts win conditions for X based on board configurations.

INDEX TERMS CART algorithm, Decision trees, Pruning, Splitting criteria, Tic-tac-toe

I. INTRODUCTION

Decision Trees are a fundamental and widely used machine learning technique for both classification and regression tasks. They operate by recursively splitting the data into subsets based on feature values, forming a tree-like model of decisions. This approach makes Decision Trees intuitive and easy to interpret, which contributes to their popularity in fields such as data science, finance, healthcare, and many others. The core idea behind Decision Trees is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features. Each internal node of the tree represents a "test" on an attribute, each branch represents the outcome of the test, and each leaf node represents a class label or continuous value (in regression tasks).

Node: A point in the tree where data is split. There are different types of nodes:

Root Node: The topmost node, representing the entire dataset, which is split into two or more subsets.

Internal Node: Represents a test on an attribute and has branches (or children).

Leaf Node: Also known as a terminal node, represents a class label (in classification) or a continuous value (in regression). It is the end of a branch in the tree.

Depth: The maximum distance from the root node to a leaf. Depth indicates the number of splits or levels in the tree and influences the model's complexity.

This hierarchical structure allows the model to capture complex patterns in the data through a series of binary decisions. Key advantages of using Decision Trees include their simplicity and interpretability, as well as their ability to handle both numerical and categorical data. Moreover, Decision Trees require little data preprocessing and can handle missing values and outliers effectively.

However, they also have some limitations, such as a tendency to overfit the training data, especially when the trees are deep, and sensitivity to small changes in the data, which can lead to entirely different splits.

Splitting Criteria: The method used to decide where to split the data at each node. Common criteria include:

Gini Impurity: Measures the frequency of different classes at a node.

Entropy: Measures the information gain from a split.

Variance Reduction: Used in regression trees to minimize the variance of the target variable within each split.

Pruning: A technique used to reduce the size of a decision tree by removing sections of the tree that provide little to no additional predictive power. Pruning helps to combat overfitting.

Feature Importance Analysis: A process to determine which features (attributes) are most influential in predicting the target variable. This is done by evaluating the impact of each feature on the tree's splits.

The primary motivation for using Decision Trees is their ability to create a clear and understandable model that can be easily visualized and interpreted. They also excel in capturing non-linear relationships and interactions between features. This lab report aims to provide a thorough exploration of Decision Trees through practical exercises. We will delve into how Decision Trees are constructed, how to implement them using popular programming libraries, and how to evaluate their performance. Through hands-on experimentation, we will gain insights into the strengths and limitations of Decision Trees and how to construct the most optimal decision tree.

II. RELATED WORK

Several popular decision tree algorithms exist, each with its strengths and weaknesses. Here are some prominent examples:

Classification and Regression Trees (CART): A foundational decision tree algorithm used for both classification and regression tasks. CART utilizes a greedy approach to select the optimal feature and split point at each node, maximizing the information gain or reduction in variance [1]. It forms the basis

for many other tree-based methods and is known for its simplicity and robustness [1].

C4.5: An extension of CART developed by Ross Quinlan, which incorporates the information gain ratio as the splitting criterion, addressing potential biases towards features with a high number of values [2]. C4.5 can handle both categorical and continuous data and introduces mechanisms for dealing with missing values and pruning trees to prevent overfitting [2].

Random Forests: An ensemble learning method that combines multiple decision trees trained on random subsets of features and data points. Random forests improve model performance by reducing variance and preventing overfitting, making them highly effective for a wide range of tasks [3]. They also provide insights into feature importance, which can be valuable for understanding the underlying data structure [3].

Gradient Boosting Machines (GBM): Another ensemble method that builds decision trees sequentially, with each tree attempting to correct the errors of the previous ones. This approach can significantly enhance the predictive accuracy of models but requires careful tuning to avoid overfitting [4].

Decision Trees in Game Endgame Analysis
Decision trees have also been utilized to analyze endgame scenarios in various games, providing insights into optimal strategies and decision-making processes. Some notable works in this area include:

Tic-Tac-Toe Endgame Analysis: Decision trees have been employed to analyze the set of all possible endgame configurations in Tic-Tac-Toe. By constructing a tree where each node represents a possible board state and each branch represents a potential move, researchers can identify optimal strategies and winning patterns. This analysis helps in understanding the game's dynamics and the importance of specific moves in determining the outcome [5].

Chess Endgames: Decision trees have been used to evaluate and predict outcomes in chess endgames. By analyzing historical game data and considering factors such as piece positions and potential moves, decision trees can assist in identifying strategies that lead to checkmate or draw scenarios. This approach has been instrumental in developing

endgame databases and training chess engines to improve their endgame play [6].

Go Endgames: In the game of Go, decision trees have been applied to analyze endgame positions and recommend moves that maximize a player's territory. Given the complexity and vast number of possible configurations, decision trees help simplify the decision-making process by breaking down the board into more manageable segments and focusing on critical areas [7].

Other Relevant Work

Credit Scoring: Financial institutions employ decision trees to assess the creditworthiness of loan applicants. By analyzing features such as credit history, income, and employment status, decision trees can provide clear and interpretable rules for determining credit risk [8].

Fraud Detection: Decision trees are instrumental in detecting fraudulent activities by analyzing transaction patterns and identifying anomalies. They offer a transparent and effective way to flag potentially fraudulent transactions for further investigation [9].

III. METHODOLOGY

A. DATASET DESCRIPTION

A decision tree is a machine learning algorithm used for classification and regression tasks, which splits data into branches to make decisions based on feature values. For the "Tic-Tac-Toe Endgame" dataset, a decision tree can be employed to predict whether a given endgame board configuration results in a win for player X, by learning patterns and rules from the dataset's attributes and class labels.

The Tic-Tac-Toe Endgame dataset consists of a snapshot of the tic-tac-toe board game at the end of each match. It contains the positions of x and o on the 9 squares of the board. The dataset consists of 958 instances where each instance represents the configuration of the tic-tac-toe board at the end of a game. The goal is to ensure a win for x assuming that x plays first. The dataset consists of data of categorical data type only with no missing values.

The target is represented by 'class' that has values positive and negative, where positive represents a win for x while negative represents a win for o. The positive value is

attained when "x" has one of 8 possible ways to create a "three-in-a-row".

The dataset consists of nine attributes that represent each square of the board such as top-left-square, top-middle-square, middle-right-square, bottom-left-square, etc. The attributes take the values x, o, and b that represent x, o, and blank values in the square.

The nine attributes of the dataset are:

1. top-left-square
2. top-middle-square
3. top-right-square
4. middle-left-square
5. middle-middle-square
6. middle-right-square
7. bottom-left-square
8. bottom-right-square
9. Bottom-right-square

B. PROPOSED METHODOLOGY

The 'tic-tac-toe' dataset used in this report contains categorical attributes and discrete outcomes making it ideal for classification tasks.

The computation of a decision tree involves several steps.

The dataset should initially be preprocessed by imputing missing values, handling categorical data, discretizing, treating outliers, normalizing, standardizing, rectifying class imbalance and selecting features and reducing dimensions.

Since, the tic-tac-toe dataset doesn't contain missing values as each cell in the dataset is either 'x', 'o' or 'b', imputation of missing values need not be performed.

Due to the presence of entirely categorical data in the dataset, the attributes are encoded into numerical values using one-hot-encoding while discretization, normalization, standardization and treating of outliers are not necessarily due to the inapplicability of categorical attribute values.

The dataset is imbalanced due the presence of significantly more instances of positive target class than negative. To address this, stratify is used during train-test split to ensure that the distribution of classes in the target variable remains consistent across both the training and testing sets which helps to maintain the same proportion of each class in both sets.

The dataset has a fixed number of features and each feature possess relevance for the decision tree formation process and

dimensionality reduction is less likely to provide any significant benefits. Therefore, feature selection is not performed for the dataset. The first step is to perform feature importance calculation to identify the most informative attributes. Next, the categorical data is converted into numerical labels using one-hot encoding. Before splitting the dataset into train set and test set, the dataset is checked for imbalanced dataset and the target class is stratified during the train-test split in case of imbalanced dataset. For balanced datasets, the dataset is then split into train and test data without stratifying the target class. After that, the training data is used to build models based on different criteria such as entropy and Gini impurity. Pruning techniques such as depth pruning (early stopping) and cost complexity pruning are performed to avoid overfitting. A confusion matrix is computed to visualize the model's performance. The trained model is then evaluated using various metrics such as accuracy, precision, recall, and F1-score. Finally, the decision tree is visualized to provide more insights into the decision-making process.

Algorithm: Decision Tree

1. Import necessary libraries
 2. Load data from CSV file
 3. Check for irrelevant data
 4. Drop irrelevant features
 5. Perform feature importance calculation
 6. Convert categorical variables to numerical labels
 7. Check for imbalanced data
 8. Stratify target class in case of imbalanced data
 9. Split the data into training and test sets
 10. Instantiate decision tree classifier using sklearn
 11. Train decision tree classifier using the training set
 12. Predict target variables for the test set
 13. Evaluate classifier performance
 14. Visualize the decision tree
-

C. MATHEMATICAL FORMULAE

The decision tree trained using Entropy and GINI using the Tic-tac-toe dataset requires the following formulas for its evaluation.

Entropy (H)

Entropy gives the measure of uncertainty or impurity associated with a random variable.

A. Shrestha and P. Dongol: Decision Tree

For a discrete random variable Y taking m distinct values $\{y_1, \dots, y_m\}$, the entropy is given as

$$H(Y) = - \sum_{i=1}^m p_i \log(p_i)$$

where $p_i = P(Y = y_i)$ represents the probability of a class i and m represents the number of classes.

Higher entropy value represents higher uncertainty and contains more information content.

Gini Impurity (G):

The Gini impurity measure is evaluated using

$$\text{Gini} = 1 - \sum_{k=1}^K p_k^2$$

where, p_k denotes the proportion of instances belonging to class k ($K = 1, \dots, k$) and $0 \log_2 0 = 0$

When a node p is split into k partitions (children), the quality of split is computed as

$$\text{GINI}_{\text{split}} = \sum_{i=1}^k \frac{n_i}{n} \text{GINI}(i)$$

where, n_i is the number of records at child i and n is the number of records at node p .

Classification Error (CE):

The classification error is the fraction of misclassified instances in a set.

For binary classification

$$\text{CE}(S) = 1 - \max(p_1, p_2, \dots, p_c)$$

where c represents the number of classes and p_i is the probability of class i .

Information Gain (IG):

Information gain measures how much information a feature gives us about a class. It measures the reduction or change in entropy achieved because of the split.

$$\text{IG}(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

where, $\text{IG}(S, A)$ represents the information gain of feature A in dataset S , $H(S)$ represents the entropy of the original dataset S , $\text{Values}(A)$ represents all the possible set of values for feature A , $|S_v|$ represents the number of samples in dataset S where feature A has value v and $H(S_v)$ is the entropy of the subset S_v where feature A has attribute v .

Gain Ratio:

The Gain Ratio is given as

$$GainRATIO_{split} = \frac{GAIN_{split}}{SplitINFO}$$

where,

$$SplitINFO = - \sum_{i=1}^k \frac{n_i}{n} \log \frac{n_i}{n}$$

The formulae used for the performance evaluation are:

Accuracy:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where, TP (True Positives) represents correctly predicted positive instances, TN (True Negatives) represents correctly predicted negative instances, FP (False Positives) represents incorrectly predicted positive instances and FN (False Negatives) represent incorrectly predicted negative instances.

Precision:

$$Precision = \frac{TP}{TP + FP}$$

Recall:

$$Recall = \frac{TP}{TP + FN}$$

F1-score:

$$F1\ Score = 2 \times \frac{Precision * Recall}{Precision + Recall}$$

where, Precision is the ratio of correctly predicted positive instances to the total predicted positive instances and Recall ratio represents the correctly predicted positive instances to the total actual positive instances.

D. INSTRUMENTATION DETAILS

The analysis and visualization in this study were conducted using Jupyter Notebook, a versatile and interactive web-based coding environment. Numeric operations and dataset manipulations were performed using the NumPy library, while the Pandas library was utilized for importing the dataset and conducting Exploratory Data Analysis (EDA). The DecisionTreeClassifier model of scikit-learn library was used to create a decision tree classifier and the sklearn library was used for train-test split, confusion matrix calculation, and the calculation of evaluation metrics. For data visualization, Matplotlib and Seaborn were employed to visualize the decision tree and confusion matrix. In summary, Jupyter Notebook, along with these powerful

libraries, facilitated efficient and effective experimentation.

IV. EXPERIMENTAL RESULTS

Feature Importance Analysis: Before performing decision tree analysis, feature importance calculation was performed. The correlation of the features with the target variable was calculated and plotted as shown in [Figure 1](#).

The features with the highest correlation with the target feature were middle_middle_o, middle_middle_x, top_left_o, top_right_o, bottom_right_o, and bottom_left_o. These features had correlation values higher than 0.10. Likewise, the features with the lowest correlation with the target features were middle_middle_b, top_middle_b, middle_right_b, middle_left_b, bottom_middle_b, bottom_right_b, top_right_b, top_left_b, and bottom_left_b. These features had correlation values lower than 0.05.

Check if the data is unbalanced: The dataset was found to be unbalanced, as illustrated in [Figure 2](#). The number of instances of the positive class was approximately double that of the negative class. So, to mitigate this class imbalance, stratification was performed while splitting the data into training and testing sets. This helps to ensure that the class proportions are maintained in both the training and testing sets and reduces the risk of the model becoming biased towards the majority class.

Initial decision tree: A decision tree was instantiated using Entropy as the splitting criterion. The decision tree was trained on the training set and predictions were subsequently made on the testing set. The performance of the decision tree was evaluated using a confusion matrix ([Figure 3](#)), a classification report ([Table 1](#)), and an ROC curve ([Figure 5](#)).

From the confusion matrix, the value of TP (True Positive) was observed to be 176 while the TN (True Negative), FP (False Positive) and FN (False Negative) were observed to be 86, 14 and 13 respectively. Similarly, from the classification report, the values of F1 score, accuracy, precision and recall were found to be 0.93, 0.90, 0.93 and 0.93 respectively.

Also, the ROC curve represents the sensitivity of the system. The True Positive Rate (TPR) represents the proportion of actual positives that are correctly identified by the model. Since the TPR value increases rapidly with the increase in FPR (False Positive Rate), which represents the proportion of actual negatives incorrectly identified as positive in the beginning, it indicates that the model has high sensitivity. Since FPR remains low for a high rise in TPR, it is considered ideal.

The decision tree was visualized, but it was found to be too complex and large to interpret effectively. To improve interpretability and mitigate overfitting, a technique called pruning can be employed. Pruning simplifies the tree by removing sections that provide little power in predicting target variables, thus making the model more generalizable and easier to understand.

Decision Tree with Early Stopping: Another decision tree was instantiated using Entropy as the splitting criterion. This time the max depth was set to a lower number (say 3). This is also known as early stopping or depth pruning. The decision tree was trained on the training set and predictions were subsequently made on the testing set. The performance of the decision tree was evaluated using a confusion matrix (Figure 7), a classification report (Table 2), and an ROC curve (Figure 8).

The values of TP, TN, FP, FN derived from the confusion matrix were found to be 163, 44, 56 and 25 respectively. Similarly, the classification report provides us with the metrics to evaluate the model like F1 score, accuracy, precision and recall. The F1 score obtained from depth pruning at depth 3 was 0.8 while the accuracy, precision and recall obtained from the same model were 0.72, 0.74 and 0.87 respectively.

Similar to the previous model, the ROC curve represents the sensitivity of the system. Since the TPR value increases slowly with the increase in FPR, it indicates that the model has low sensitivity. Since the graph remains closer to the reference line, it represents that the performance of the model has degraded with respect to the model without pruning.

The decision tree was visualized as illustrated in Figure 9. Here, we can see that the **root node** is **'middle_middle_o'**. The root node

consists of different values that play an important role in splitting the data at the root node. **'middle_middle_o <= 0.5'** refers to the condition used to split the data at the root node. Here, **0.5** is the threshold used for deciding how to split the data in the root node. If the feature **'middle_middle_o'** is less than or equal to 0.5, then the decision tree follows the left branch (**True branch**). Likewise, if the feature **'middle_middle_o'** is greater than 0.5, then the decision tree follows the right branch (**False branch**). **'entropy 0.931'** gives the entropy value at the root node, which is the measure of impurity or disorder at the node. The entropy value ranges from 0 to 1, where 0 indicates that all samples are of one class (pure node), and 1 indicates a 50-50 split between two classes (maximum impurity). An entropy value of **0.931** suggests a fairly mixed set of class labels, meaning there's a significant amount of disorder or uncertainty about the class of the samples in this node. **'samples = 670'** indicates the total number of samples contained in the root node. So, there are 670 samples from the dataset being considered at the root node. **'value = [232.0, 438.0]'** represents the distribution of samples across the different classes. It shows the counts of samples in each class (**[No, Yes]**). It means that 232 samples belong to the class labeled 'No' (i.e. negative class or class '0') and 438 samples belong to the class labeled 'Yes' (i.e. positive class or. class '1'). **'class = Yes'** indicates the majority class at the root node. Since there are more samples labeled "Yes" (438) compared to those labeled "No" (232), the majority class at this node is "Yes". The decision tree uses this majority class to classify samples that reach this node if no further splitting improves the classification.

Calculations for the root node values:

We have a total of **958** instances in the dataset. The train-test split was performed with a train size of **0.7**. So, the total number of samples in the node size is initially:

Number of samples = $0.7 * 958 = 670$

Out of the 670 samples, the number of instances labeled 'No' = **232**

Similarly, the number of instances labeled 'Yes' = **438**

Since stratify was used, the class proportions are maintained in the train set. In the dataset, the class imbalance led to the No to Yes ratio

being **1:2** approximately. This can be reflected in the training set i.e. 232/438 is approximately **1:2**.

Entropy calculation is done as shown:

$$E = -\frac{232}{670} \log_2 \left(\frac{232}{670} \right) - \frac{438}{670} \log_2 \left(\frac{438}{670} \right) = 0.931$$

The root node is chosen as **middle_middle_o** because it leads to the split with the highest information gain. Also, this is evident from the fact that **middle_middle_o** has the highest correlation with the target feature when feature importance analysis was done.

For splitting, the condition used is **middle_middle_o <= 0.5**. In the dataset, **428** out of **670** samples satisfy this condition. So, they are split into the left child of the root node. Similarly, **268** samples fail to satisfy the condition. So, they are split into the right child of the root node.

Similarly, other nodes can be explained as follows:

In the left child of the root node, the decision tree evaluates the feature '**bottom_left_o**' to decide how to further split the data. With an entropy of **0.759**, there's still a significant mix of both classes. There are **428** samples in this node. Out of **428** samples, **94** are labeled 'No' and **334** are labeled 'Yes'. The majority class is **Yes**, so the class is labeled as **Yes**.

In the right child of the root node, the decision tree evaluates the feature '**top_right_x**' to decide how to further split the data. It has an entropy of **0.986**, which is more than that of the root node. There are **242** samples in this node. Out of **242** samples, **138** are labeled 'No' and **104** are labeled 'Yes'. The majority class is **No**, so the class is labeled as **No**.

In the left child of the '**bottom_left_o <= 0.5**' node, the decision tree evaluates the feature '**top_right_o**' to decide how to further split the data. With an entropy of **0.599**, there's relatively less mix of both classes. There are **268** samples in this node. Out of **268** samples, **39** are labeled 'No' and **229** are labeled 'Yes'. The majority class is **Yes**, so the class is labeled as **Yes**.

In the right child of the '**bottom_left_o <= 0.5**' node, the decision tree evaluates the feature '**top_left_o**' to decide how to further split the data. It has an entropy of **0.928**, which indicates a significant

amount of disorder. There are **160** samples in this node. Out of **160** samples, **55** are labeled 'No' and **105** are labeled 'Yes'. The majority class is **Yes**, so the class is labeled as **Yes**.

In the left child of the '**top_right_x <= 0.5**' node, the decision tree evaluates the feature '**bottom_left_x**' to decide how to further split the data. The entropy value is **0.916**. There are **124** samples in this node. Out of **124** samples, **83** are labeled 'No' and **41** are labeled 'Yes'. The majority class is **No**, so the class is labeled as **No**.

In the right child of the '**top_right_x <= 0.5**' node, the decision tree evaluates the feature '**top_left_x**' to decide how to further split the data. It has an entropy of **0.997**, which indicates a significant amount of disorder. There are **118** samples in this node. Out of **118** samples, **55** are labeled 'No' and **63** are labeled 'Yes'. The majority class is **Yes**, so the class is labeled as **Yes**.

In the left child of the '**top_left_o <= 0.5**' node, the entropy value is 0. This means that the split has resulted in a node with the highest purity i.e. all classes belonging to a single class. Out of **154** samples, all 154 are labeled 'Yes'. The majority class is **Yes**, so the class is labeled as **Yes**. This is a leaf node, so no further splitting is required.

In the right child of the '**top_left_o <= 0.5**' node, the entropy value is **0.927**. Out of **114** samples, **39** are labeled 'No' and **75** are labeled 'Yes'. The majority class is **Yes**, so the class is labeled as **Yes**. This is a leaf node, so no further splitting is required. So, any instance that ends up here is predicted to belong to 'Yes' class.

Similarly, other node values of the remaining leaf nodes can be evaluated.

Finding the optimal depth: The decision tree obtained without pruning had excellent performance, but was prone to overfitting and was difficult to interpret due to its complexity. On the other hand, the decision tree obtained by early stopping at a depth of 3 was easy to visualize and interpret, but the performance was not satisfactory due to being very simple. So, to find the optimal depth of the decision tree, the F1 score was plotted against tree depth/complexity as shown in [Figure 4](#). The plot revealed that the highest F1 score was achieved with a tree depth of 6. Therefore, the optimal depth of our decision tree is 6.

Decision Tree with Optimal Depth: With splitting criterion as Entropy and tree depth pruned to optimal depth i.e. a depth of 6, another decision tree was instantiated. It was trained on the training set and predictions were subsequently made on the testing set. The performance of the decision tree was evaluated using a confusion matrix (Figure 10), a classification report (Table 3), and an ROC curve (Figure 11).

The confusion matrix obtained at the optimal depth provided True Positive and True Negative values of 181 and 84 respectively while the False Positive and False Negative values were 16 and 7 respectively.

From the ROC curve, it was observed that the rate of increase of TPR with respect to FPR increased rapidly in the beginning which represented increased sensitivity of the model.

The decision tree was visualized as illustrated in Figure 12.

Cost Complexity Pruning: Using cost complexity pruning, a decision tree was instantiated with splitting criterion as Entropy. The optimal alpha value was computed using cross-validation. Similar to previous methods, the decision tree was trained on the training dataset and the test dataset was used to evaluate the performance of the model. A classification report (Table 4) was computed and the confusion matrix (Figure 13) as well as the ROC curve (Figure 14) were visualized for model evaluation.

The classification report provided insight on the F1 score, accuracy, precision, and recall, the values of which were observed to be 0.93, 0.9, 0.92, and 0.94 respectively. The calculation of these metrics is assisted by the data obtained from the confusion matrix, i.e., TP=176, TN=84, FP=16, and FN=12. Finally, the ROC curve provided information about the sensitivity of the system. Since there is a rapid increase in TPR with respect to FPR in the beginning, the model is approaching the perfect classifier. Thus, it indicates that the model has high sensitivity.

Decision Tree using GINI splitting criteria: To train a decision model using GINI, initially, a decision tree was instantiated using GINI as the splitting criterion. The provided dataset was split into train set and test set. The train set was used to train the decision tree while the test set was used to test the performance of the model on metrics such as F1

score, accuracy, precision and recall. To evaluate the model, confusion matrix (Figure 16), classification report (Table 5) as well as an ROC curve (Figure 17) were used.

Once trained, the test data was used to predict its classes. Based on the predicted class and the actual class, a confusion matrix was computed having TP (True Positive) value 177, TN (True Negative) value 86, FP (False Positive) value 14 and FN (False Negative) value 11. These values work as a base for calculation of the evaluation metrics obtained from the classification report, the values of which were observed to be F1-score 0.93, accuracy 0.91, precision 0.93 and recall 0.94. Similar to the ROC curve obtained using Entropy, the TPR increases rapidly in the beginning and the rate gradually slows down reaching an almost ideal state. This indicated that the model was highly sensitive.

The decision tree formed using GINI criterion resembled that of the Entropy criterion. Calculations for the root node values is almost same as that for Entropy. Only calculation of GINI is different:

$$GINI = 1 - \left[\left(\frac{232}{670} \right)^2 + \left(\frac{438}{670} \right)^2 \right] = 0.453$$

After splitting based on the condition **middle_middle_o <= 0.5**, 428 out of 670 samples satisfy this condition. So, they are split into the left child of the root node. Similarly, 268 samples fail to satisfy the condition. So, they are split into the right child of the root node.

$$GINI_{SPLIT} = \left(\frac{428}{670} \right) GINI_1 + \left(\frac{242}{670} \right) GINI_2$$

Where,

In the left child node, out of 428 samples, 94 are labeled 'No' and 334 are labeled 'Yes'.

$$GINI_1 = 1 - \left[\left(\frac{94}{428} \right)^2 + \left(\frac{334}{428} \right)^2 \right] = 0.343$$

In the right child node, out of 242 samples, 138 are labeled 'No' and 104 are labeled 'Yes'.

$$GINI_2 = 1 - \left[\left(\frac{138}{242} \right)^2 + \left(\frac{104}{242} \right)^2 \right] = 0.490$$

So,

$$GINI_{SPLIT} = 0.396$$

V. DISCUSSION AND ANALYSIS

From feature importance analysis, we can gain key insights about the nature of the game. Among the features with a high correlation with the target feature, a common trend that we can notice is that most of these features have 'o' value. This finding signifies that the chance of winning is higher for players that start first in tic-tac-toe i.e. play as 'o'. Similarly, among the features with a low correlation with the target feature, a common trend that we can notice is that all these features have 'b' values. It indicates that blank values have no role in determining the outcome of the game, which is obvious.

Let's compare and analyze the performance of our first two decision trees: the Initial Decision Tree (max depth) and the Second Decision Tree (Early Stopping at depth = 3).

Initially, the performance of the decision tree with maximum depth appeared strong, achieving approximately 90% accuracy, recall, and F1 score. However, this could be attributed to overfitting, given the tree's deep and complex structure. To address this, pruning was applied by stopping early at depth 3. Unfortunately, this led to a notable decline in performance (accuracy dropped to 72%), as the simplified tree struggled to capture the dataset's intricacies. Nonetheless, it greatly facilitated visualization and interpretation. To strike a balance between performance and generalization, we determined the optimal depth for pruning. The third decision tree was pruned to this optimal depth (depth = 6), surpassing the initial tree's performance while mitigating overfitting risks. This optimized tree not only exhibited superior performance and generalization but also enhanced visualization and interpretability with reduced complexity.

Let's compare and analyze the performance of decision trees using two pruning techniques: Depth pruning (Early Stopping) and Cost complexity pruning.

First, with depth pruning, the decision tree achieved outstanding results when pruned to an optimal depth of 6. It delivered an accuracy of 92%, with precision, recall, and F1

scores ranging between 85% to 95%. Next, employing cost complexity pruning with the optimal alpha value yielded similar performance. The decision tree also attained 90% accuracy, with precision, recall, and F1 scores consistently between 85% to 95%. In terms of computational efficiency, depth pruning typically involves examining fewer nodes and stopping the growth of the tree early, which can reduce time complexity compared to cost complexity pruning, where calculating the optimal alpha value through cross-validation may require more computational resources. While both pruning techniques demonstrated comparable overall performance, depth pruning slightly outperformed cost complexity pruning in this scenario.

Let's compare and analyze the performance of decision trees using two splitting criteria: Entropy and GINI.

The decision tree demonstrated consistent performance regardless of whether Entropy or GINI was used as the splitting criterion. Both criteria yielded similar results in terms of accuracy, precision, recall, and F1 score. In terms of efficiency, both Entropy and GINI are computationally efficient for splitting nodes in decision trees. However, GINI index is slightly faster to compute compared to Entropy because it involves only the calculation of probabilities squared, whereas Entropy involves logarithmic calculations. This minor difference in computation time usually does not significantly impact overall performance unless dealing with very large datasets or deep trees. Overall, whether using Entropy or GINI, the decision tree exhibited comparable performance metrics, making either criterion a viable choice depending on specific dataset characteristics and interpretability preferences.

VI. CONCLUSION

In conclusion, decision trees stand as powerful tools in machine learning, enabling intuitive decision-making by partitioning data based on feature attributes. This study specifically aimed to construct decision trees modeling the decision-making process in tic-tac-toe, focusing on scenarios where the player acts as X. We explored various decision tree configurations, adjusting parameters such as

splitting criteria (Entropy vs. GINI), pruning techniques (depth pruning vs. cost complexity pruning), and conducting feature importance analysis to enhance performance. Our findings emphasize that depth pruning generally yields slightly superior results in terms of accuracy and other metrics, while both Entropy and GINI criteria offer comparable effectiveness in capturing optimal strategies. Efficiency considerations underscore the balance between computational resources and model robustness, crucial for real-time applications. Ultimately, optimizing these factors is essential for developing decision tree models that provide reliable predictions and insights across diverse machine learning applications.

APPENDIX

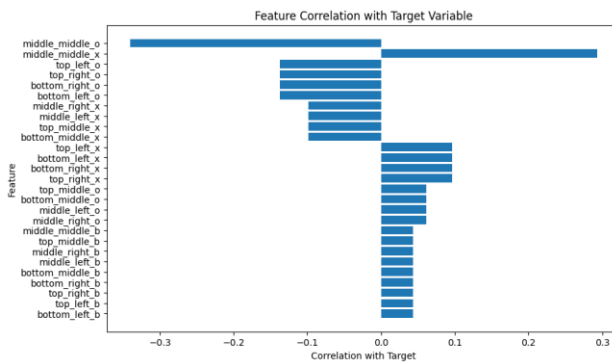


Figure 1 Feature Correlation with Target Variables

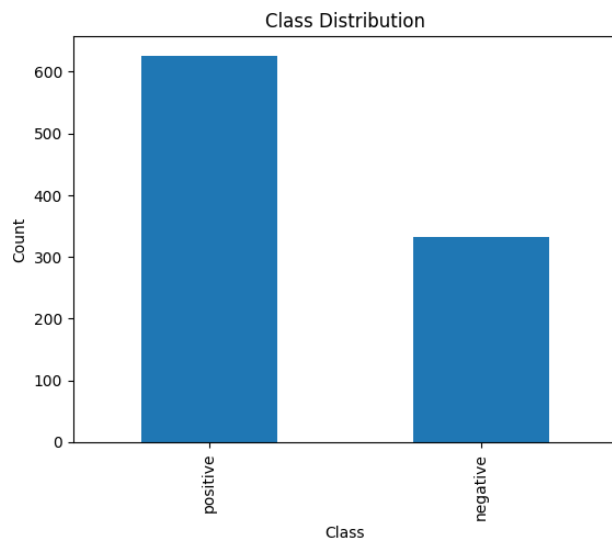


Figure 2 Bar Graph of Target Class Distribution

A. Shrestha and P. Dongol: Decision Tree

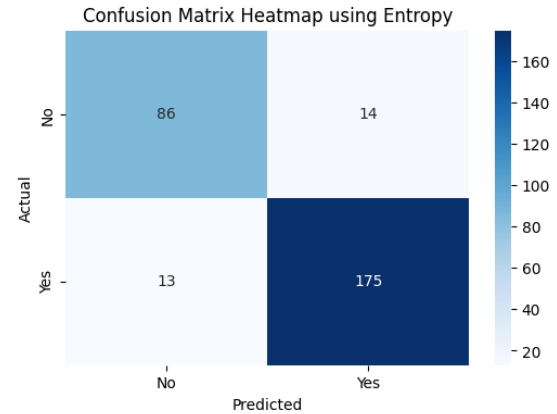


Figure 3 Confusion Matrix using Entropy

Table 1 Classification Report of Decision Tree using Entropy

	precision	recall	F1-score	support
Negative	0.87	0.86	0.86	100
Positive	0.93	0.93	0.93	188
Accuracy			0.91	288
Macro avg	0.90	0.90	0.90	288
Weighted avg	0.91	0.91	0.91	288

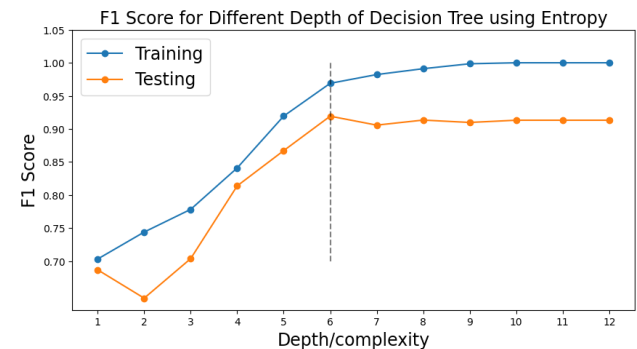


Figure 4 F1 Score for Different Depths of Decision Tree using Entropy

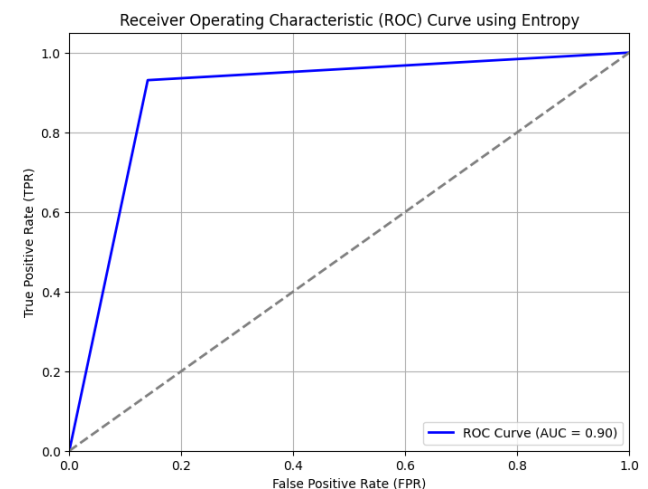


Figure 5 ROC Curve using Entropy

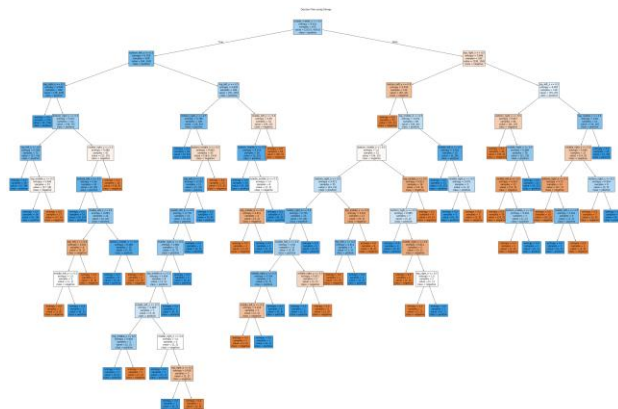


Figure 6 Decision Tree using Entropy at max depth (Hard to visualize and interpret)

Confusion Matrix Heatmap using Early Stopping (depth=3)

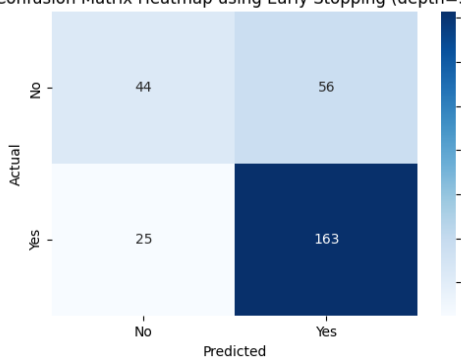


Figure 7 Confusion Matrix using Early Stopping at depth 3

Table 2 Classification Report of Decision Tree using Early Stopping at depth 3

	pre- ci- sion	recall	F1-score	Support
Negative	0.64	0.44	0.52	100
Positive	0.74	0.87	0.8	188
accuracy			0.72	288
Macro avg	0.69	0.65	0.66	288
Weighted avg	0.71	0.72	0.70	288

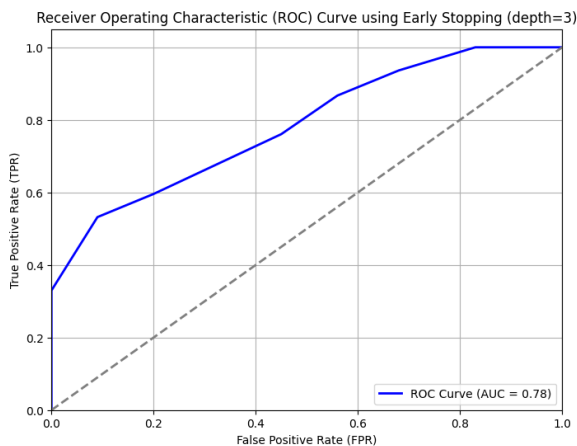


Figure 8 ROC Curve using Early Stopping at depth 3

A. Shrestha and P. Dongol: Decision Tree

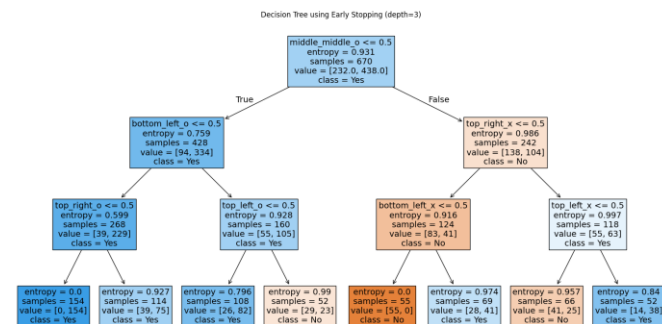


Figure 9 Decision Tree using Early Stopping at depth 3

Confusion Matrix Heatmap using Early Stopping (optimal depth=6)

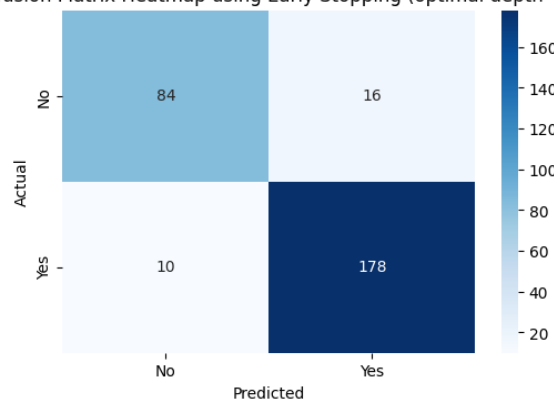


Figure 10 Confusion Matrix using Early Stopping at optimal depth

Table 3 Classification Report using Early Stopping at optimal depth

	precision	recall	F1-score	support
Negative	0.89	0.84	0.87	100
positive	0.92	0.95	0.93	188
Accuracy			0.91	288
Macro avg	0.91	0.89	0.90	288
Weighted avg	0.91	0.91		288

Receiver Operating Characteristic (ROC) Curve using Early Stopping (optimal depth=6)

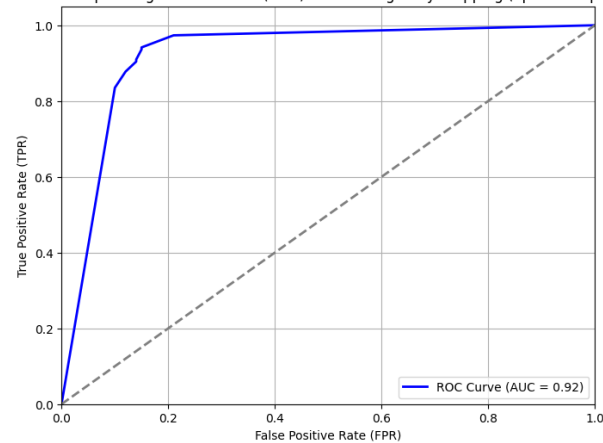


Figure 11 ROC Curve using Early Stopping at optimal depth

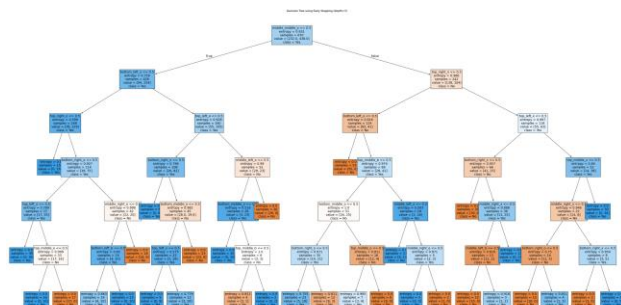


Figure 12 Decision Tree using Early Stopping at optimal depth

Confusion Matrix Heatmap using Cost Complexity Pruning

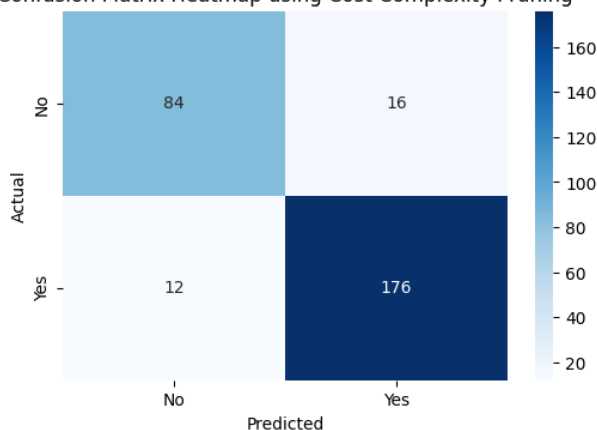


Figure 13 Confusion Matrix using Cost Complexity Pruning

Table 4 Classification Report using Cost Complexity Pruning

	pre- ci- sion	recall	F1-score	Support
Negative	0.88	0.84	0.86	100
Positive	0.92	0.94	0.93	188
Accuracy			0.90	288
Macro avg	0.90	0.89	0.89	288
Weighted avg	0.90	0.90	0.90	288

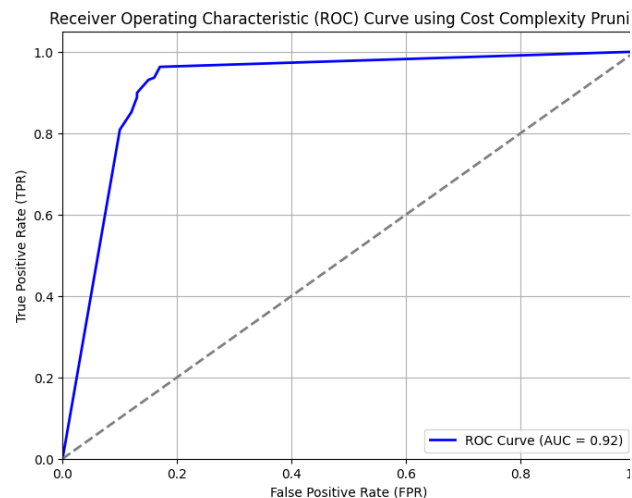


Figure 14 ROC Curve using Cost Complexity Pruning

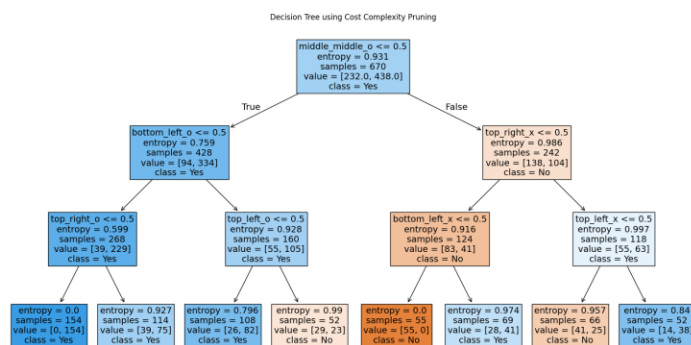


Figure 15 Decision Tree using Cost Complexity Pruning

Confusion Matrix Heatmap using GINI

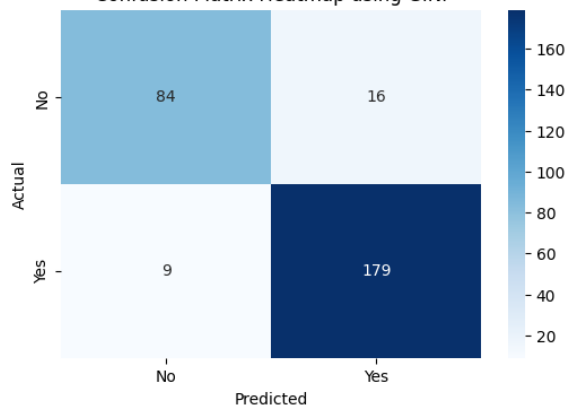


Figure 16 Confusion Matrix using GINI pruned at optimal depth

Table 5 Classification Report using GINI Pruned at optimal depth

	pre- ci- sion	recall	F1-score	Support
Negative	0.90	0.84	0.87	100
Positive	0.92	0.95	0.93	188
Accuracy			0.91	288
Macro avg	0.91	0.90	0.90	288
Weighted avg	0.91	0.91	0.91	288

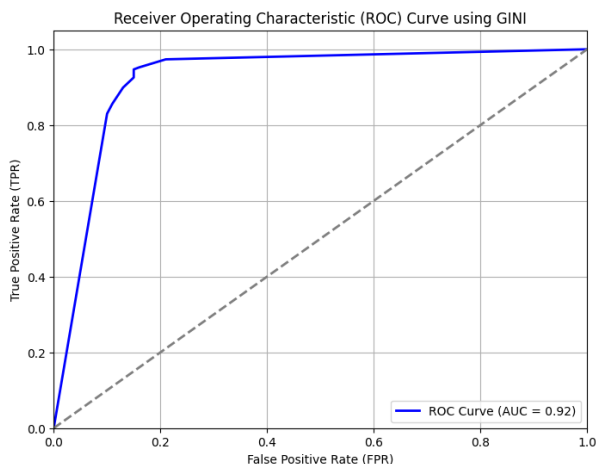


Figure 17 ROC Curve using GINI Pruned at optimal depth

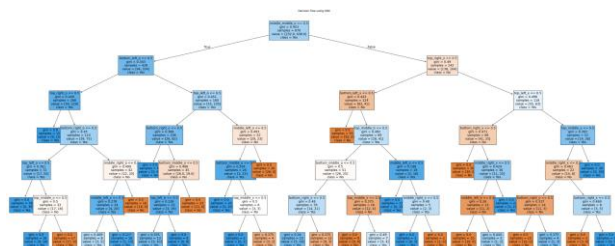


Figure 18 Decision Tree using GINI pruned at optimal depth

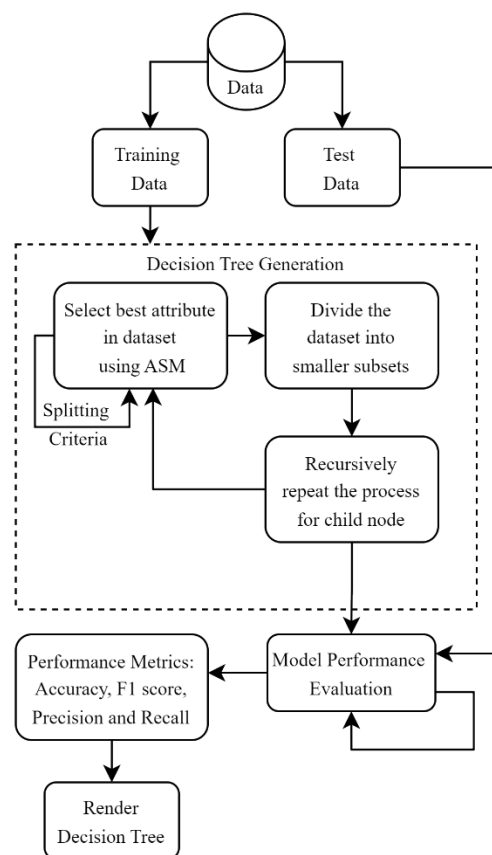


Figure 19 System Architecture of Decision Tree

ACKNOWLEDGEMENT

We extend our heartfelt appreciation to the Department of Electronics and Computer Engineering at IOE, Thapathali Campus for providing us with the invaluable opportunity to engage in learning, research, and the realization of our insights. We would like to express our sincere gratitude to our esteemed lecturer Er. Dinesh Baniya Kshatri for providing unwavering support, invaluable guidance, and insightful suggestions throughout our academic career.

A. Shrestha and P. Dongol: Decision Tree

REFERENCES

- [1] L. Breiman, J. Friedman, R. A. Olshen and C. J. Stone, "Classification and Regression Trees," *CRC Press*, p. 368, 1984.
- [2] J. R. Quinlan, "C4.5: Programs for Machine Learning," *Morgan Kaufmann Publishers*, p. 302, 1993.
- [3] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5-32, 2001.
- [4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189-1232, 2001.
- [5] D. Michie, D. J. Spiegelhalter and C. C. Taylor, "Machine Learning, Neural and Statistical Classification," 1994.
- [6] D. C. Wilkins, "Using knowledge of Chess Endgames to learn how to play chess," *AAAI Conference on Artificial Intelligence*, 1980.
- [7] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, V. D. G. Drissi and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7585, pp. 484-489, 2016.
- [8] L. C. Thomas, "A survey of credit and behavioral scoring: Forecasting financial risk of lending to consumers," *International Journal of Forecasting*, vol. 16, no. 2, pp. 149-172, 2000.
- [9] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235-255, 2002.

BIOGRAPHY



Aayush Man Shrestha is currently pursuing his undergraduate degree in Computer Engineering at the Institute of Engineering (IOE), Thapathali Campus. His research interests span across diverse areas of ML and AI, including natural language processing, computer vision, robotics, and automation.



Pradeepti Dongol is currently pursuing her undergraduate degree in Computer Engineering at the Institute of Engineering (IOE), Thapathali Campus. Her research interests span across diverse areas, including natural language processing, generative AI, arts and design, and Large Language Models (LLMs).